

Travail de Bachelor

Pipeline distribué d'annotation automatique d'images géospatiales

Accélération de l'IA par segmentation des ressources GPU

Étudiant	Dani Tiago Faria dos Santos
Superviseure	Prof. Bertil Chapuis
Département	Technologies de l'information et de la communication (TIC)
Filière	Informatique et systèmes de communication (ISC)
Orientation	Réseaux et systèmes (ISC-RS)
Entreprise mandante	Prof. Bertil Chapuis HEIG-VD — Institut IICT Route de Cheseaux 1 1400 Yverdon-les-Bains
Année académique	2025-26

Yverdon-les-Bains, le 08.05.2026

Table des matières

Authentification	5
Préambule	6
Resumé	7
1 Introduction	8
1.1 Contexte	8
1.2 Problème	8
1.3 Objectifs	9
2 État de l’art	10
2.1 Segmentation d’images	10
2.2 Calcul distribué	11
2.3 Stockage objet	14
2.4 Format de résultats	14
2.5 Annotation	16
2.6 Observabilité	16
2.7 Cycle vie d’un log	18
3 Architecture	19
3.1 Vue d’ensemble	19
3.2 Infrastructure Kubernetes	19
3.3 RayCluster	21
3.4 Cache du modèle SAM3	21
3.5 Stratégie de tuilage	21
3.6 Format de sortie Parquet	21
3.7 Intégration Label Studio	22
3.8 Observabilité	23

4 Résultats	24
5 Conclusion	25
Bibliographie	26
Annexes	29
I Glossaire	30

Authentification

Par la présente, j'atteste avoir réalisé ce travail et n'avoir utilisé aucune autre source que celles expressément mentionnées.

Dani Tiago Faria dos Santos

Yverdon-les-Bains, le 08.05.2026

Préambule

Ce travail de Bachelor (ci-après TB) est réalisé en fin de cursus d'études, en vue de l'obtention du titre de Bachelor of Science HES-SO en Ingénierie.

En tant que travail académique, son contenu, sans préjuger de sa valeur, n'engage ni la responsabilité de l'auteur, ni celles du jury du travail de Bachelor et de l'Ecole.

Toute utilisation, même partielle, de ce TB doit être faite dans le respect du droit d'auteur.

HEIG-VD

Le Chef de département TIC

Yverdon-les-Bains, le 08.05.2026

Resumé

Travail de Bachelor 2025-26

Titre: Pipeline distribué d'annotation automatique d'images géospatiales

Sous-titre: Accélération de l'IA par segmentation des ressources GPU

Ce travail porte sur la conception et le déploiement d'un pipeline distribué d'annotation automatique d'images géospatiales. Le pipeline exploite SAM3 pour la segmentation et Ray pour distribuer le traitement sur les GPUs du cluster Kubernetes de la HEIG-VD. Les images sont lues depuis le service S3 sur MinIO, découpées en patches et traitées en parallèle par des workers Ray. Les métadonnées GPS extraites de l'EXIF sont associées aux polygones produits, stockés au format Parquet sur S3. Une couche d'observabilité basée sur Prometheus, Loki et Grafana permet de surveiller l'état du cluster, ses performances et d'identifier les incidents.

Étudiant	Dani Tiago Faria dos Santos
Superviseure	Prof. Bertil Chapuis
Entreprise mandante	Prof. Bertil Chapuis

1 Introduction

1.1 Contexte

L’Institut IICT de la HEIG-VD conduit le projet NearAI, dont l’objectif est de construire une base de données géospatiale d’éléments routiers à partir d’acquisitions mobiles. Les images sont capturées par des véhicules équipés de systèmes différents comme par exemple un Trimble MX50 ou d’une goPro Max, qui produisent des panoramas équirectangulaires allant jusqu’à 8’192 x 4’096 pixels accompagnés de coordonnées GPS enregistrées dans l’EXIF. Le jeu de données cible actuel dépasse 300’000 images.

Annoter manuellement ce volume n’est pas faisable, un annotateur humain habile et expérimenté consacrant trente secondes par image mettrait plus de 2’500 heures pour couvrir l’ensemble du corpus. L’annotation automatisée est la seule voie viable.

1.2 Problème

Plusieurs classes d’objets sont ciblées, comme par exemple, les panneaux de signalisation (`sign`) ou encore les marquages au sol (`road_marking`). Pour chaque image, le pipeline doit produire un ensemble de polygones identifiant ces objets, avec, leur classe, leur score de confiance, leurs coordonnées GPS et leurs dimensions normalisées.

Ces polygones transitent ensuite vers une plateforme de traitement d’image, où, des annotateurs humains corrigent et valident les prédictions.

Chaque image à 8’192 x 4’096 pixels dépasse la fenêtre d’entrée de tout modèle de segmentation courant. Il faut donc découper l’image en tuiles avant l’inférence. Ensuite, traiter 300’000 images en un délai raisonnable impose de distribuer le calcul sur plusieurs GPUs en parallèle.

1.3 Objectifs

1.3.1 Traitement des images

Ce travail conçoit et déploie un pipeline distribué couvrant les étapes suivantes :

1. Lecture des images depuis le bucket S3.
2. Découpage en tuiles 512 x 512 pixels et inférence sur chaque tuile.
3. Extraction des polygones, normalisation des coordonnées et association des métadonnées GPS issues de l'EXIF.
4. Écriture des résultats au format Parquet sur le bucket.
5. Import des pré-annotations dans Label Studio pour validation humaine.

Tout cela s'exécute sur le cluster Kubernetes de la HEIG-VD via le framework Ray, qui distribue les tâches GPU sur les workers disponibles.

1.3.2 Côté utilisateur

Une API servira de porte d'accès aux développeurs ou utilisateurs du service pour faciliter l'accès aux services batch ou on-demand.

1.3.3 Analyse de la pipeline

La pipeline sera annaylable au niveau de ses performances et état via un dashboards alimenté de métriques, logs et traces.

2 État de l'art

2.1 Segmentation d'images

SAM3 (Segment Anything Model 3) est le modèle de segmentation publié par Meta en 2024 [1]. Il hérite de SAM2 et étend ses capacités aux images de haute résolution et à la vidéo. SAM3 adopte une architecture Vision Transformer (ViT) comme encodeur d'image et un décodeur léger qui produit des masques binaires à partir de prompts géométriques (points, boîtes, polygones).

Le modèle fonctionne en mode *promptable* et en mode *everything mode*.

En mode **prompt**, il attend simplement un message comme : « Un panneau octogonal avec le texte STOP à son centre », ou, simplement, « Un panneau ». Tandis que en mode **everything**, il segmente tous les objets détectables de l'image. Aucune supervision n'est fournie à l'inférence et SAM3 propose des masques candidats que le pipeline filtre par classe et score.

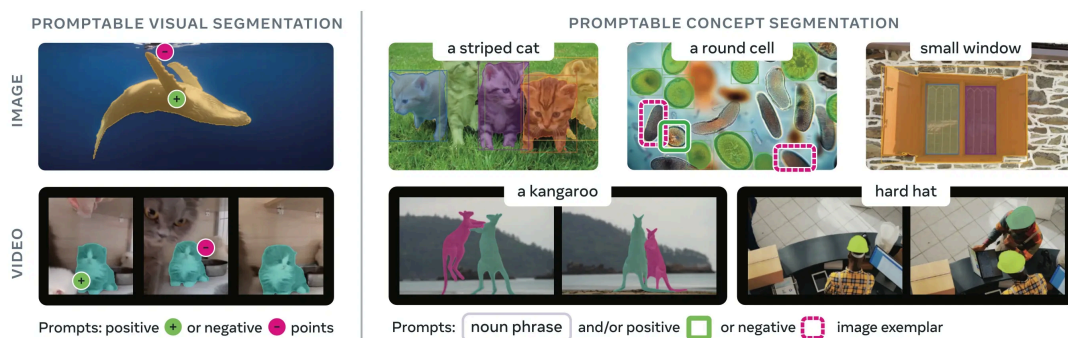


Fig. 1. – Exemple du mode prompt de SAM3. Source : <https://docs.ultralytics.com/fr/models/sam-3/>

SAM3 a été entraîné sur SA-1B, un corpus de 1,1 milliard de masques sur 11 millions d'images. Cette couverture lui confère une généralisation forte sur des domaines non vus à l'entraînement, dont les images routières équirectangulaires.

Ce modèle accepte des images jusqu'à 1'024 x 1'024 pixels. Une panoramique de 8'192 x 4'096 pixels doit donc être découpée avant l'inférence. Ce travail adopte des tuiles de 512 x 512 pixels, ce qui produit 128 tuiles par image à pleine résolution. Un downsampling à 50 % ramène ce nombre à 32 tuiles et réduit le temps d'inférence d'un facteur 4, au prix d'une perte de détail acceptable pour les classes cibles.

NB : SAM3 redimensionne automatiquement grandes les images sur une résolution autour de 1024 px, donc il n'y pas d'intérêt d'aller au-delà.

Les images équirectangulaires présentent une distorsion géométrique croissante vers le zénith et le nadir. Les objets cibles (panneaux, marquages) se concentrent dans la bande centrale de l'image, correspondant à $\pm 30^\circ$ d'élévation, là où la distorsion est minimale. La correction de projection n'est donc pas implémentée, elle apporterait un gain marginal pour un coût d'implémentation élevé. Le bas du panorama est en grande partie occulté par la carrosserie du véhicule. Ce choix est documenté comme limitation connue.

2.2 Calcul distribué

Apache Spark est le framework de calcul distribué dominant pour les workloads analytiques sur données structurées. Son modèle d'exécution repose sur un DAG de transformations sur des RDDs ou DataFrames, optimisé pour les opérations SQL et les pipelines ETL à large échelle sur clusters homogènes.

Spark présente trois limitations structurelles pour l'inférence GPU :

Héritage CPU : Spark a été conçu dans l'écosystème Hadoop pour le traitement de données tabulaires. Le support GPU a été ajouté a posteriori via RAPIDS (NVIDIA). Il n'est pas natif : les workers Spark ne savent pas scheduler dynamiquement des tâches GPU hétérogènes.

Clusters homogènes : Spark optimise pour la localité des données sur des clusters uniformes. Le cluster iict-rad dispose de trois types de GPU (L40S, A40, L4) avec des performances très différentes. Ray gère nativement cette hétérogénéité via ses mécanismes de placement group et de priorité par ressource.

Performance sur inférence ML : Sur des benchmarks de classification d'images en batch, Ray Data atteint une vitesse 2x supérieure à Spark [2]. Sur des workloads multimodaux (images, vidéo), l'écart est 10x par rapport à Spark [3].

Le signal industriel le plus fort est la migration d'Amazon en 2024 : leur équipe Business Data Technologies a migré 1,5 exaoctets de données Parquet de Spark vers Ray, réalisant une économie de plus de 120 millions de dollars par an avec une efficacité 82 % supérieure par GiB

traité [4]. Cette migration a pris 4 ans et constitue la plus grande validation publique de Ray en production.

Ray est un framework Python open-source pour distribuer des workloads ML/IA sur des clusters hétérogènes CPU/GPU [5]. Il expose deux primitives fondamentales.

- Une **Task** (`@ray.remote` sur une fonction) est une unité de calcul sans état, exécutée de façon asynchrone sur un worker disponible.
- Un **Actor** (`@ray.remote` sur une classe) est une unité de calcul avec état, maintenu en mémoire sur un worker assigné. L'Actor est LE patron adapté au chargement de modèles. Le modèle est chargé une fois dans `__init__`, puis réutilisé sur toutes les requêtes sans rechargement, évitant les erreurs de mémoire insuffisante (OOM) qui surviennent lorsqu'un modèle est rechargé à chaque tâche.

```
1 @ray.remote(num_gpus=1)
2 class SAM3Worker:
3     def __init__(self):
4         self.model = load_sam3()    # chargé une fois
5
6     def process(self, batch):
7         return infer(self.model, batch)
```

python

Liste 1. – L'Actor initialise le modèle, le preprocesseur et le postprocesseur une seule fois dans `__init__` ; `process()` les réutilise sur chaque image sans rechargement

Ray gère l'allocation des ressources (CPU, GPU, mémoire), la sérialisation des données entre driver et workers via son object store partagé et la récupération sur défaillance d'un worker.

Concernant le choix entre ces deux technologies, Spark reste pertinent pour les pipelines ETL sur données structurées. Pour de l'inférence GPU sur images non structurées à grande échelle sur un cluster hétérogène, Ray est le choix correct.

Deux stratégies de parallélisation GPU existent pour l'inférence de modèles :

- **Data parallelism** : chaque GPU héberge une copie complète du modèle et traite une image indépendante. Le throughput scale linéairement avec le nombre de GPU.
- **Model parallelism** : le modèle est fragmenté sur plusieurs GPU pour réduire la latence d'une seule inférence. Implique un overhead de communication inter-GPU à chaque couche.

Le model parallelism est justifié uniquement lorsque le modèle ne tient pas sur un seul GPU (LLMs de 70 milliards de paramètres et plus). SAM3 ViT-H occupe 2,4 Go de VRAM et les GPU du cluster (L40S 48 Go, A40 48 Go, L4 24 Go) l'hébergent sans contrainte. L'objectif du pipeline est le throughput sur 300 000 images, pas la latence sur une image isolée. Avec N workers en data parallelism, N images sont traitées simultanément sans aucune synchronisation inter-GPU. Le modèle Ray Actor (un modèle chargé par GPU, N workers indépendants) est la stratégie optimale pour ce workload.

KubeRay est l'opérateur Kubernetes officiel pour Ray [6]. Il introduit la ressource **RayCluster** (CRD `ray.io/v1`), qui déclare un nœud head et un ou plusieurs groupes de workers. L'opérateur crée et gère les pods correspondants, expose les ports GCS (6379), dashboard (8265), métriques (8080) et client Ray (10001) via des Services Kubernetes.

Le driver externe se connecte au cluster via `ray.init("ray://ray-cluster-head-svc:10001")`. Les workers ne sont pas contactés directement car Ray dispatche les tâches via le Global Control Store (GCS) hébergé sur le head.

Une distinction importante sépare la terminologie Kubernetes de la terminologie Ray :

Terme	Sens Kubernetes	Sens Ray
Head	Pas d'équivalence directe	Nœud unique hébergeant le GCS, le scheduler et le dashboard
Worker	Pod dans un Deployment	Nœud Ray enregistré auprès du GCS, exécute les tâches
Service	ClusterIP stable pour la sélection de pods	Expose GCS (6379), dashboard (8265), client (10001)

Tableau 1. – Terminologie Kubernetes vs Ray

2.3 Stockage objet

Les images panoramiques (JPEG, 50 Mo chacune), les fichiers Parquet et les poids de modèles sont des **BLOBs**, des objets binaires nonstructurés que les bases de données relationnelles gèrent mal et que les systèmes de fichiers partagés (NFS) ne scalent pas. Le stockage objet est conçu pour ce cas. Chaque BLOB est adressé par une clé unique, accessible via HTTP, sans limite de taille ni de volume total. **S3** est le protocole standard pour ce type de stockage. boto3, PyArrow et Ray Data l'implémentent nativement, aucune couche d'abstraction supplémentaire n'est nécessaire. Enfin, remplacer la variable `S3_ENDPOINT_URL` suffit à changer de backend (MinIO, AWS S3, Ceph) sans modifier le code du pipeline.

MinIO est un serveur de stockage objet compatible avec le protocole S3 [7]. La HEIG-VD l'exploite sur un NAS Synology SA3200D, installé via Container Manager avec l'image officielle minio/minio. Le pipeline accède à MinIO via boto3 avec les variables d'environnement S3 standard (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `S3_ENDPOINT_URL`).

Des alternatives comme **RustFS** (Apache 2.0, moins d'un an) ou **CEPH** (mature, LGPL, bloc/fichier/objet) ont été évaluées. Malgré le fait que MinIO devient un produit fermé dès cette année, la décision est de conserver MinIO car la migration ajouterait du risque sans bénéfice immédiat, et la configuration S3 est le seul point à modifier si un changement de backend devient nécessaire.

2.4 Format de résultats

Parquet est un format binaire orienté colonnes conçu pour les workloads analytiques [8]. Chaque fichier est découpé en *row groups* eux-mêmes découpés en *column chunks*. Cette organisation permet de lire uniquement les colonnes nécessaires à une requête sans charger l'ensemble du fichier.

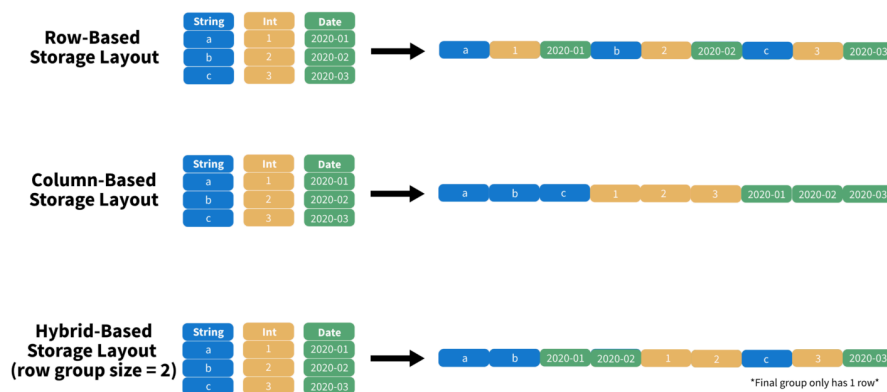


Fig. 2. – Parquet exploite une solution hybride pour obtenir de meilleures performances. Source : <https://towardsdatascience.com/demystifying-the-parquet-file-format-13adb0206705/>

Parquet stocke des statistiques min/max par column chunk. Un moteur de requête peut ainsi ignorer des row groups entiers si le prédicat tombe hors de leur plage. C'est le *predicate pushdown*, supporté nativement par PyArrow. Filtrer les polygones par zone GPS ou par seuil de score ne charge que les colonnes `latitude`, `longitude` et `score`, indépendamment du volume total.

PostGIS (extension PostgreSQL pour les données géospatiales) a été évalué comme alternative. Il offre des index spatiaux GIST et des requêtes géométriques natives (`ST_Within`, `ST_Intersects`). La décision est de le remplacer par Parquet sur S3. Les requêtes du projet ne nécessitent pas de jointures géospatiales complexes, et Parquet évite de maintenir une base de données.

JSON (JavaScript Object Notation) est le format accepté par LabelStudio pour importer les données géospatiales sur une image. Ce format va être utilisé uniquement pour le mode on demand et en legacy pour labelstudio afin de visualiser les résultats des runs.

```
1  [{
2    "data": {
3      "image": "s3://nearai/data/acquisitions/.../image.jpg"
4    },
5    "predictions": [{
6      "model_version": "SAM3",
7      "result": [{
8        "type": "polygonlabels",
9        "from_name": "label",
10       "to_name": "image",
11       "original_width": 8192,
12       "original_height": 4096,
13       "value": {
14         "closed": true,
15         "polygonlabels": ["sign"],
16         "points": [[42.05, 40.72], ...]
17       }
18     }]
19   }]
```

```
20 }]
```

Liste 2. – Exemple d'output JSON pour une image traitée et marquée sur LabelStudio

2.5 Annotation

Label Studio est une plateforme d'annotation open-source [9]. Elle supporte les polygones (`polygonlabels`) sur images, le stockage S3 comme source d'images, et l'import de pré-annotations via son API REST. Le pipeline produit des pré-annotations SAM3 que des annotateurs humains corrigent et valident dans Label Studio. Il est configuré avec MinIO comme *cloud storage* source.

Les URLs `s3://nearai/...` sont converties en URLs HTTP temporaires signées, ce qui évite d'exposer les credentials de stockage aux navigateurs clients.

XXXX Parler ici de NearLabel, le projet fait par mon collègue Valentin Ricard.

2.6 Observabilité

Prometheus permet de faire du monitoring et des alertes via la collection de métriques en temps réel. Les données sont stockées sous format TSDB (Time Series DataBase). Une fois récoltées, les données peuvent être traitées via PromQL, un langage query extrêmement rapide se basant sur les vecteurs instantanés, de portée et vecteurs scalaires.

Ses principaux composants sont:

- **Core Server** : Service principal de prometheus configuré via le fichier *prometheus.yaml*.
- **Exporters** : Agents qui collectent les métriques tel que Node Exporter, Database Exporter.
- **Alertmanager** : Pour la gestion d'alertes avec des fonctions de groupe ou d'inhibition.

Grafana est un outil permettant de créer et visualiser des dashboards via sa web UI. Couplé à Prometheus pour la récolte de données, des visualisations en temps réel peuvent être effectuées pour monitorer le status des différents nœuds et composants de la pipeline.

Loki est un système d'agrégation de journaux qui indexe uniquement les balises de métadonnées, et non le contenu complet des journaux. Cela le rend plus léger qu'Elasticsearch, ce qui est utile dans notre cas, car nous n'avons pas besoin d'une recherche en texte intégral, mais devons plutôt retrouver les journaux d'un pod de travail Ray spécifique à un moment précis. Il stocke les données dans un format S3 et est composé de 2 types de stockages principaux *index* et *chunks*.

- **Index** : Table des matières, mappe simplement les labels sur la position des chunks.
- **Chunks** : Blocs compressés de logs bruts selon un label et une plage horaire.

Cette chaine permet a Loki d'être purement *stateless* car toutes les données sont stockées sur le service S3. Les logs sont traités avec LogQL (langage fortement inspiré de PromQL) en les filtrant selon les labels pour les transformer ensuite en métriques.

```
1 {namespace="dani", pod=~"ray-worker.*"} |= "ERROR"
```

LogQL

Liste 3. – Selectionne uniquement les logs « ERROR » du namespace dani sur les pods commençant par ray-worker

Promtail est un agent supplémentaire pour Loki. Il tourne en tant que *DaemonSet* avec 1 instance par noeud et collecte toutes les métriques des pods sur ce dit noeud. Les pods sont découverts via le *service discovery* de l'API K8s puis en lisant dans `/var/logs/pods/`. Les workers Ray dans notre cas n'auront qu'à écrire dans leur *stdout/stderr* et Promtail les récupères.

Malheureusement, Promtail a été mis en EOL. Il faut donc utiliser **Alloy** comme substitut.

Grafana Alloy est le successeur officiel de Promtail et de Grafana Agent, annoncé en maintenance-only en 2024 [10]. Il unifie la collecte de logs, métriques, traces et profils dans un seul agent configurable. Sa configuration utilise le langage River, un format déclaratif inspiré de HCL. Les composants sont connectés explicitement. C'est à dire que la sortie d'un bloc devient l'entrée du suivant, formant un pipeline de traitement.

```
1 discovery.kubernetes "pods" { role = "pod" }
```

alloy

```
2
```

```
3 loki.source.kubernetes "pods" {
```

```
4     targets    = discovery.kubernetes.pods.targets
```

```
5     forward_to = [loki.write.loki.receiver]
```

```
6 }
```

```
7
```

```
8 loki.write "loki" {
```

```
9     endpoint { url = "http://loki-svc:3100/loki/api/v1/push" }
```

```
10 }
```

Liste 4. – Le composant `loki.source.kubernetes` lit les logs directement via l'API Kubernetes sans monter le système de fichiers du nœud

Les avantages par rapport à Promtail sont que un seul pod **Deployment** suffit à la place d'un **DaemonSet** et les limites inotify du nœud ne sont pas sollicitées. Car si Promtail lisait chaque fichier de logs et *stdout/stderr*, Alloy lui va simplement scanner les sources à intervalles réguliers et donc ainsi ne consomme pas de ressource "inotify".

2.7 Cycle vie d'un log

Fonctionnement en 3 étapes :

1. **Ingestion** : Les logs sont récoltés
2. **Mise en tampon** : Le tout est push sur Loki
3. **Flush vers S3** : Au bout d'un certain temps ou une certaine taille de fichier les logs sont envoyés sur le bucket s3.

2.7.1 DCGM

3 Architecture

3.1 Vue d'ensemble

Le pipeline suit un flux linéaire en cinq étapes :

1. **Lecture** : le driver liste les objets du préfixe S3 d'entrée et distribue les clés d'image aux workers Ray.
2. **Téléchargement** : chaque worker télécharge l'image depuis MinIO et extrait les coordonnées GPS de l'EXIF.
3. **Inférence** : l'image est découpée en tuiles 512×512 px (après downsampling optionnel), chaque tuile est passée à SAM3 en mode *everything*.
4. **Agrégation** : les masques produits sont convertis en polygones, normalisés en pourcentage des dimensions de l'image originale, filtrés par score de confiance.
5. **Écriture** : les polygones sont sérialisés dans un fichier Parquet et envoyés sur MinIO.



Fig. 3. – Vue d'ensemble du pipeline

3.2 Infrastructure Kubernetes

Le pipeline s'exécute sur le cluster `iict-rad` de la HEIG-VD (Kubernetes 1.33.9). Le cluster expose 9 GPUs répartis sur trois nœuds :

Nœud	GPUs	Modèle	VRAM
iict-suchet	3	NVIDIA L40S	46 GB
iict-k8s-node4-rad	2	NVIDIA A40	46 GB
iict-chasseron	4	NVIDIA L4	23 GB

Tableau 2. – GPUs disponibles sur le cluster `iict-rad`

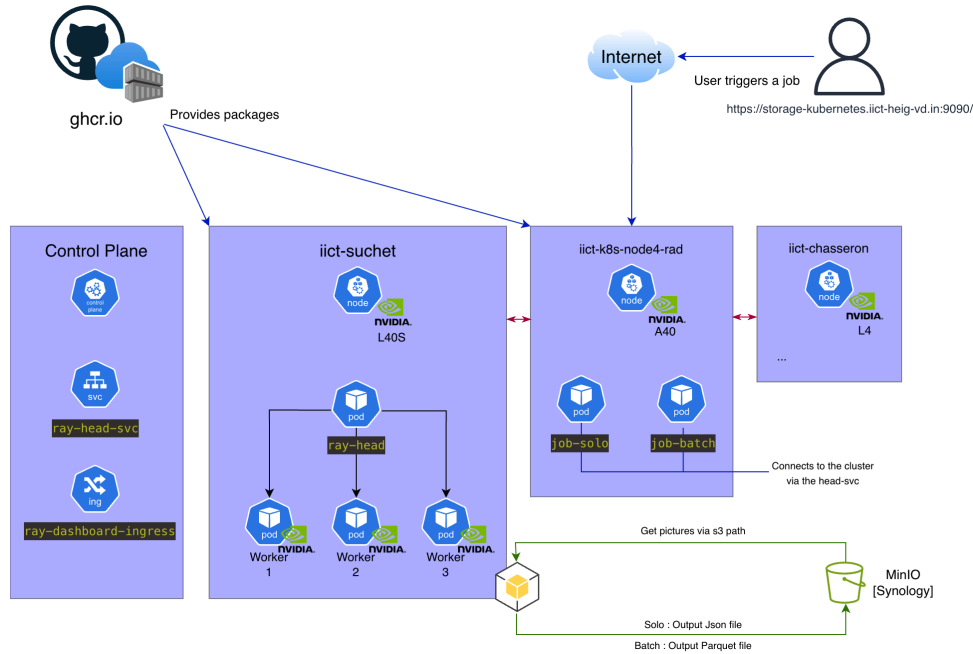


Fig. 4. – Les différentes ressources k8s sont distribuables sur n’importe quel noeud, à l’exception des workers qui visent des noeuds avec GPU.

Les workers Ray ciblent exclusivement les nœuds L40S et A40 via `nodeAffinity`. Le nœud `iict-chasseron` est exclu pour deux raisons :

- ses L4 offrent une puissance de calcul inférieure (23 GB VRAM vs 46 GB)
- et il portait un taint `node.kubernetes.io/disk-pressure` lors des tests.

Ce taint, appliqué automatiquement par Kubernetes quand l’usage disque franchit un seuil, expose les pods à des évictions SIGTERM sans préavis.

Le GPU Operator NVIDIA est installé sur le cluster. Il installe automatiquement les drivers GPU, le device plugin et DCGM Exporter sur chaque nœud. Les requêtes de ressource `nvidia.com/gpu` fonctionnent sans configuration manuelle.

3.3 RayCluster

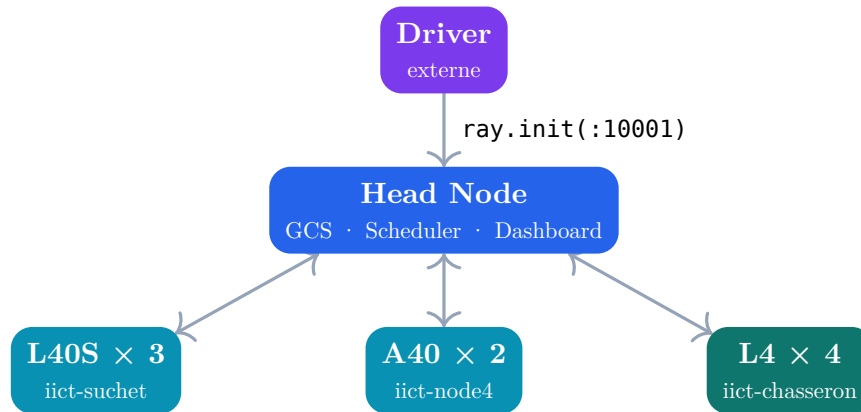


Fig. 5. – Architecture du RayCluster sur iict-rad

3.3.1 Injection des credentials

3.4 Cache du modèle SAM3

3.5 Stratégie de tuilage

3.6 Format de sortie Parquet

Chaque image produit un fichier Parquet nommé `<acquisition_id>/<image_stem>.parquet`, écrit sur le préfixe S3 de sortie. Le schéma est le suivant :

Colonne	Type	Description
image_key	string	Chemin S3 complet de l'image source
acquisition_id	string	Dossier parent (identifiant de l'acquisition)
label	string	sign ou road_marking
score	float32	Score de confiance SAM3 (0–1)
points	string	Polygone JSON encodé, en % des dimensions de l'image
original_width	int32	Largeur de l'image source en pixels
original_height	int32	Hauteur de l'image source en pixels
latitude	float64	Latitude GPS en degrés décimaux (null si absent)
longitude	float64	Longitude GPS en degrés décimaux (null si absent)

Tableau 3. – Schéma Parquet de sortie du pipeline

La compression Snappy est appliquée. Les fichiers sont interrogeables directement depuis DuckDB ou PyArrow sans étape de chargement en base de données.

3.7 Intégration Label Studio

Label Studio est connecté à MinIO comme *source storage* S3. Les URLs `s3://nearai/...` sont converties en URLs HTTP temporaires pré-signées, servies directement du stockage au navigateur sans proxy.

L'interface de labeling XML définit deux classes :

```
1 <View>
2   <Image name="image" value="$image"/>
3   <PolygonLabels name="label" toName="image">
4     <Label value="sign" background="#FF0000"/>
5     <Label value="road_marking" background="#FFFF00"/>
6   </PolygonLabels>
7 </View>
```



Les pré-annotations importées doivent utiliser `from_name`: "label" et `to_name`: "image" pour correspondre à cette interface. Une divergence de noms provoque l'affichage des polygones sans label (gris).

3.8 Observabilité

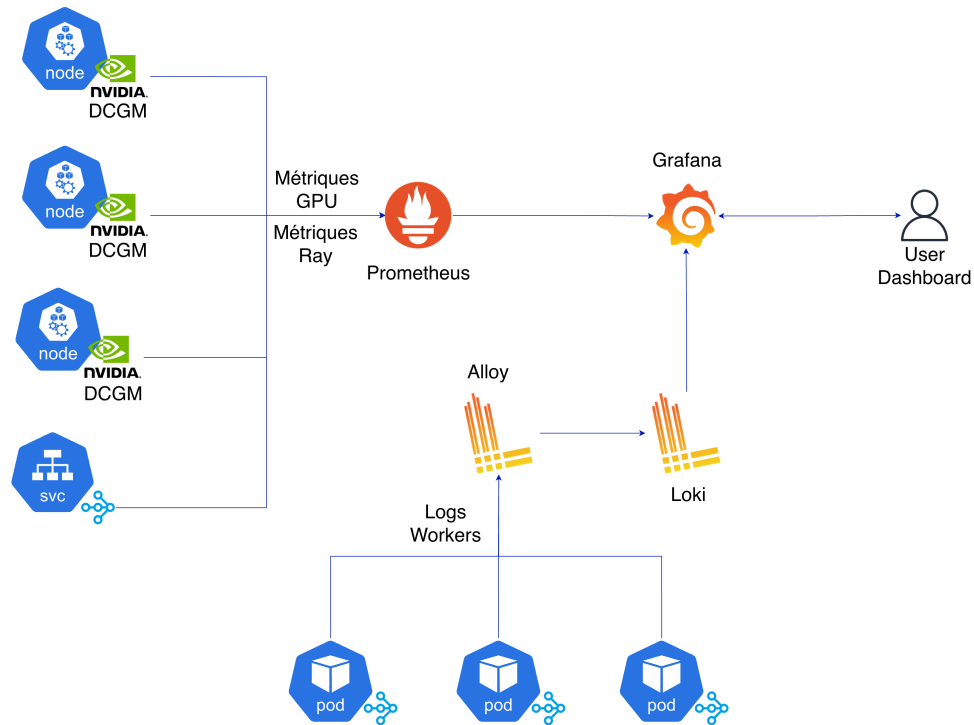


Fig. 6. — Prometheus récolte les métriques tandis que Loki s'occupe des logs afin d'alimenter Grafana.

4 Résultats

5 Conclusion

Bibliographie

- [1] Nikhila Ravi *et al.*, « SAM 2: Segment Anything in Images and Videos », 2024. [En ligne]. Disponible sur: <https://arxiv.org/abs/2408.00714>
- [2] Anyscale, « Ray Data vs Apache Spark: A Benchmark Comparison ». [En ligne]. Disponible sur: <https://www.anyscale.com/blog/ray-data-vs-apache-spark>
- [3] Eventual Inc., « Distributed ML Data Processing Benchmarks: Daft, Ray Data, Spark, and Modin ». [En ligne]. Disponible sur: <https://blog.getdaft.io/p/distributed-ml-data-processing-benchmarks>
- [4] Rahul Pandey, Bo Xiong, et Anyscale, « How Amazon Migrated 1.5 Exabytes from Apache Spark to Ray ». [En ligne]. Disponible sur: <https://www.anyscale.com/blog/how-amazon-business-migrated-from-apache-spark-to-ray>
- [5] Philipp Moritz *et al.*, « Ray: A Distributed Framework for Emerging AI Applications », 2018. [En ligne]. Disponible sur: <https://arxiv.org/abs/1712.05889>
- [6] Ray Project, « KubeRay: Kubernetes Operator for Ray ». [En ligne]. Disponible sur: <https://ray-project.github.io/kuberay/>
- [7] MinIO Inc., « MinIO High-Performance Object Storage ». [En ligne]. Disponible sur: <https://min.io/docs/minio/linux/index.html>
- [8] Apache Software Foundation, « Apache Parquet ». [En ligne]. Disponible sur: <https://parquet.apache.org/docs/>
- [9] HumanSignal, « Label Studio: Data Labeling Software ». [En ligne]. Disponible sur: <https://labelstud.io/>
- [10] Grafana Labs, « Grafana Alloy: OpenTelemetry Collector ». [En ligne]. Disponible sur: <https://grafana.com/docs/alloy/latest/>

Table des figures

Fig. 1	Exemple du mode prompt de SAM3. Source : https://docs.ultralytics.com/fr/models/sam-3/	10
Fig. 2	Parquet exploite une solution hybride pour obtenir de meilleur performances. Source : https://towardsdatascience.com/demystifying-the-parquet-file-format-13adb0206705/	14
Fig. 3	Vue d'ensemble du pipeline	19
Fig. 4	Les différentes ressources k8s sont distribuables sur n'importe quel noeud, à l'exception des workers qui visent des noeuds avec GPU.	20
Fig. 5	Architecture du RayCluster sur iict-rad	21
Fig. 6	Prometheus récolte les métriques tandis que Loki s'occupe des logs afin d'allimenter Grafana.	23

Liste des tableaux

Tableau 1 Terminologie Kubernetes vs Ray	13
Tableau 2 GPUs disponibles sur le cluster iict-rad	19
Tableau 3 Schéma Parquet de sortie du pipeline	21
Tableau 4 Glossaire	30

Annexes

I Glossaire

Terme	Définition
Kubernetes	Système d'orchestration de conteneurs open-source
KubeRay	Opérateur Kubernetes pour déployer des clusters Ray
Ray	Framework Python de calcul distribué, optimisé pour les workloads GPU
SAM3	Segment Anything Model v3 : modèle de segmentation d'images de Meta AI
ZARR	Format N-dimensionnel orienté chunking, évalué comme optimisation optionnelle
EXIF	Métadonnées embarquées dans les fichiers image, contenant notamment les coordonnées GPS
MinIO	Serveur de stockage objet compatible S3
Parquet	Format de stockage colonnaire optimisé pour les requêtes analytiques
Label Studio	Plateforme open-source d'annotation de données pour le machine learning
Pod	Unité de déploiement atomique dans Kubernetes
S3	Protocole de stockage objet défini par Amazon Web Services
GPU	Graphics Processing Unit
Actor	Abstraction Ray permettant de charger un modèle une fois et de le réutiliser sur plusieurs tâches
Prometheus	Système de collecte de métriques par scraping, modèle pull
DCGM Exporter	Exporteur NVIDIA exposant les métriques GPU vers Prometheus
Promtail	Agent de collecte de logs, tourne sur chaque node K8s

Terme	Définition
Loki	Agrégateur de logs compatible S3, intégré à Grafana
Grafana	Interface de visualisation des métriques et des logs
EOL	End of life, mise en terminaison d'un produit ou d'une solution
GCS	Global Control Service, Control Plane de Ray
Control Plane	Cerveau d'un système réseau ou bien d'un système distribué. Décide et contrôle la façon dont les données sont processées par les data plane
Data Plane	Partie de l'infrastructure qui est responsable pour la transmissions de donnée/paquets
DAG	Directed Acyclic Graph, système de séquences pour optimiser l'exécution en étapes non cycliques permettant un traitement parallèle efficace et avec une tolérance aux pannes
BLOB	Binary Large Object, fichier binaire non structuré (image JPEG, poids de modèle). Stocké dans un système de stockage objet (S3/MinIO) plutôt qu'en base de données
Alloy	Successeur de Promtail (Grafana Labs). Agent de collecte unifié pour logs, métriques et traces. Configuré via le langage River. Remplace le DaemonSet par un Deployment unique
River	Langage déclaratif de configuration d'Alloy, inspiré de HCL. Les composants sont connectés explicitement : la sortie d'un bloc devient l'entrée du suivant
ViT	Vision Transformer, architecture d'encodeur d'image basée sur l'attention, utilisée par SAM3 pour encoder les images en représentations latentes
SA-1B	Dataset d'entraînement de SAM, 1,1 milliard de masques sur 11 millions d'images. Confère à SAM3 une généralisation forte sur des domaines non vus
VRAM	Video RAM, mémoire dédiée d'un GPU. SAM3 ViT-H occupe 2,4 Go de VRAM
OOM	Out of Memory, erreur de dépassement de mémoire. Survient si un modèle est rechargé à chaque tâche au lieu d'être maintenu dans un Actor

Terme	Définition
Data parallelism	Stratégie de parallélisation GPU où chaque GPU héberge une copie complète du modèle et traite une image indépendante. Le throughput scale linéairement avec le nombre de GPU
Model parallelism	Stratégie de parallélisation GPU où le modèle est fragmenté sur plusieurs GPU pour réduire la latence d'une seule inférence. Justifié uniquement quand le modèle ne tient pas sur un seul GPU
Tuile	Découpage d'une image panoramique en sous-images 512×512 px pour l'inférence SAM3, qui est limité à 1024×1024 px en entrée
Équirectangulaire	Projection cartographique des images panoramiques 360° . Produit une distorsion géométrique croissante vers le zénith et le nadir
RayCluster	CRD Kubernetes introduit par KubeRay (ray.io/v1). Déclare un nœud head et des groupes de workers. L'opérateur gère les pods correspondants
Head Node	Nœud Ray unique hébergeant le GCS, le scheduler et le dashboard. Point d'entrée du cluster via <code>ray.init()</code>
Task (Ray)	Unité de calcul sans état dans Ray (<code>@ray.remote</code> sur une fonction). Exécutée de façon asynchrone sur un worker disponible
DaemonSet	Ressource Kubernetes qui déploie exactement un pod par nœud du cluster. Utilisé par Promtail pour collecter les logs sur chaque machine
Deployment	Ressource Kubernetes gérant des pods répliquables avec rolling updates et rollback. Utilisé par Alloy en remplacement du DaemonSet
Headless Service	Service Kubernetes sans ClusterIP. La résolution DNS retourne les IPs de tous les pods ciblés, permettant à Prometheus de scraper chaque instance individuellement
NodeAffinity	Règle de scheduling Kubernetes contraignant ou préférant certains nœuds pour un pod, basée sur les labels des nœuds (ex. type de GPU)
TSDB	Time Series DataBase, format de stockage de Prometheus optimisé pour les métriques horodatées
PromQL	Langage de requête de Prometheus. Opère sur des vecteurs instantanés, de portée et scalaires pour agréger les métriques
LogQL	Langage de requête de Loki, fortement inspiré de PromQL. Filtre les logs par labels puis les transforme en métriques

Terme	Définition
Snappy	Algorithme de compression rapide utilisé par Parquet. Prioritise la vitesse de décompression sur le taux de compression
inotify	Mécanisme Linux de surveillance des modifications de fichiers. Consommé massivement par Promtail qui ouvre un watcher par fichier de log
Spark	Apache Spark, framework de calcul distribué dominant pour les workloads ETL sur données structurées. Conçu pour clusters homogènes CPU
RAPIDS	Bibliothèque NVIDIA ajoutant le support GPU à Spark. Non natif : ne gère pas le scheduling dynamique de tâches GPU hétérogènes
ETL	Extract Transform Load, pipeline d'extraction, transformation et chargement de données. Cas d'usage principal d'Apache Spark

Tableau 4. – Glossaire